Ders 1 Genişletilmiş Notu: Lojik Kapılar ve Boole Cebri Tekrarı, Multiplexer, Decoder ve Encoder

Ders 1 Genişletilmiş Notu: Lojik Kapılar ve Boole Cebri Tekrarı, Multiplexer, Decoder ve Encoder

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Konusu ve Kapsamı
- 2. Lojik Kapılar
- 3. Lojik Fonksiyonların Gösterimi ve Çoğunluk Fonksiyonu
- 4. Boole Cebri
- 5. Karnaugh Haritası ile İndirgeme
- 6. Sayı Sistemleri ve İkiye Tümleyen
- 7. Multiplexer
- 8. Decoder
- 9. Demultiplexer
- 10. Encoder
- 11. Priority Encoder
- 12. Soru 1: Decoder ve AND-OR Dizisi ile 8'den 1'e Multiplexer
- 13. Soru 2: Decoder ve OR Kapılarıyla Üç Fonksiyon
- 14. Soru 3: 8'den 1'e Multiplexer ve Bir Inverter
- 15. Soru 4: 4'ten 1'e Multiplexer ve Harici Kapılar

1. Dersin Konusu ve Kapsamı

Bilgisayar Donanımı, Lojik Devreler dersinin devamı gibi düşünülebilir. Hocaya göre bu ders bizi bilgisayara ve bilgisayarın iç yapısına biraz daha yaklaştırır. Bir bilgisayarın işlemcisini, bellek birimlerini, çevre birimlerini ve portlarını, bunların birbiriyle nasıl haberleştiğini yavaş yavaş öğreneceğiz. Dersi anlamak için lojik devreleri biliyor olmak gerekir, çünkü bu dersin temelini lojik devreler oluşturur. Bu yüzden ilk iki üç haftada lojik konuları, yalnız bu derste gerekecek temel kavramlar üzerinden, çok hızlı tekrar edilecek.

Dönem boyunca işlenecek konular sırasıyla şunlardır:

- **Kombinasyonel devreler:** Bu haftanın konusu.
- **Ardışıl (sequential) lojik:** Flip-floplar ve bunlarla yapılan tasarımlar.
- **Saklayıcılar ve sayıcılar:** Register ve counter yapıları.
- **Programlanabilir yapılar:** Programmable logic device (PLD) çeşitleri.
- **Bellekler:** Random Access Memory ve Read Only Memory. Biri uçucudur, yani kaydedilen veriyi saklamaz. İç yapıları ve nasıl genişletildikleri ayrıntılı işlenecek.
- **Saklayıcılar arası veri transferi ve ortak veri yolu:** Bir merkezi işlem biriminin içinde veri yolu ve adres yolu gibi ortak yol (bus) yapıları vardır. Bunların teknik olarak nasıl oluşturulduğu görülecek. Burada low level programming dersinden tanıdık olabilecek adresleme yöntemleri ve komut yapıları da ele alınacak.
- **Merkezi işlem birimi (CPU) tasarımı:** Aritmetik lojik birim ve bunun saklayıcılarla haberleşmesi.

- **Pipeline veri işleme:** Veriyi paralel olarak nasıl işleyebileceğimiz.
- **Giriş/çıkış:** İşlemcinin portlarla nasıl haberleştiği.
- **Önbellek (cache memory):** Üç eşleme (mapping) tekniği olan direct mapped, set associative ve associative yapılar, epey örnekle.
- **Sanal bellek (virtual memory):** Paging ve sayfa tabloları.

  Hocanın vurgusu: Dönemin ilk yarısı lojik devreleri bildiğimiz için çok kolay gelecek. Son yarısında ise gerçekten yeni kavramlara odaklanılacak ve dersin merkezi (core) kısmı burasıdır. Bu konular iyi kavranırsa sonraki İşletim Sistemleri dersindeki bellek yönetimi (memory management) yapısı da daha rahat anlaşılır.

Dersin kaynağı, Mano'nun *Logic and Computer Design Fundamentals* kitabıdır (derste 5. sürümü kullanılıyor). Bu kitap, Lojik Devreler dersinde kullanılan kitabın bir ileri versiyonudur. İlk 5-6 bölümü lojik anlatır, sonraki bölümleri bilgisayar tasarımına (computer design) geçer. Hoca dilinin kolay, örneklerinin çok olduğunu söyleyerek kitabı tavsiye etti. Önceki sürümlerde de aynı konular anlatılıyor ve derste kitabın bölüm sonu problemleri üzerinden ilerlenecek.

2. Lojik Kapılar

2.1 AND ve OR Kapılarını Bir Cümleyle Tanımlamak

Hoca tekrara bir soruyla başladı: AND kapısının çalışma mantığını, matematiksel bir ifadeyle değil, bir cümleyle nasıl anlatırsınız? Sınıftan pek çok cevap geldi. "Çarpma işlemi" cevabını hoca kabul etmedi. "En az iki şartla gerçekleşen durumlar" cevabına da aynı şeyin OR için de söylenebileceğini, ayrıca üç girişli kapıda da AND'den söz edildiğini hatırlatarak itiraz etti. "Bir şeyi ya tam yaparsın ya hiç yapmazsın" ve "bir doğru bütün yanlışları götürür" gibi esprili cevaplar da geldi. "Girişlerin hepsi 1 ise çıkış 1" doğru bir teknik tanımdır, ama hocanın aradığı cümle şuydu:

AND: Girişlerden bir tanesi bile 0 ise çıkış 0 olur. Yani bir tane 0 hepsini sıfırlar.

OR için benzer tanım şöyledir:

OR: Girişlerden bir veya birden fazlası 1 ise çıkış 1 olur.

Hocanın buradaki amacı, bir kapının fonksiyonunu tek cümleyle tarif edebilmemizdir.

2.2 NOT, NAND ve NOR

NOT kapısı girişin tümleyenini (komplementini) alır. NAND, bir AND kapısının arkasına NOT kapısı eklenerek oluşturulur. NOR ise bir OR kapısının arkasına NOT kapısı eklenerek oluşturulur. Her kapının bir de grafik sembolü vardır. Bunlar lojik dersinden hatırlanan standart sembollerdir.

2.3 XOR Kapısının Genel Tanımı

Hoca aynı soruyu XOR için sordu. Sınıftan gelen cevap "girişler farklı olduğunda 1, aynı olduğunda 0" oldu. Hoca bunun iki giriş için çok güzel bir tanım olduğunu söyledi, ama hemen şu soruyu ekledi: İki değil de üç, dört giriş olduğunda ne diyeceğiz? Bu yüzden ikiden fazla giriş için de geçerli, daha genel bir tanım verdi:

XOR: Girişlerdeki 1'lerin sayısı tek ise çıkış 1'dir.

İki girişli doğruluk tablosunda tek sayıda 1 içeren iki durum (01 ve 10) vardır ve çıkış tam bu iki durumda 1'dir. Üç girişli XOR'un doğruluk tablosu da aynı tanımla yazılır.

2.4 XOR Kapısı ve Eşlik (Parity)

Hoca ardından even parity ve odd parity kavramlarını hatırlattı ve sordu: XOR kapısı hangi parity üretecini (parity generator) gerçekleştirir? Bir öğrenci doğru cevabı verdi: even parity.

Gerekçesi şudur. Even parity'de, üretilen parity biti dahil, 1'lerin sayısı çift olmalıdır. XOR ise girişlerdeki 1'lerin sayısı tekse 1 üretir. Girişte tek sayıda 1 varsa üretilen 1 de eklenince

toplam çift olur. Girişte çift sayıda 1 varsa XOR 0 üretir ve toplam zaten çifttir. Sonuçta XOR kapısı bir even parity generator olarak çalışır. Böylece basit bir kapıdan anlamlı bir devreye geçmiş oluruz.

> **Hocanın vurgusu:** XOR kapısıyla ilgili bu anlatılanlar akılda kalsın. XOR önemli bir fonksiyondur.

2.5 Tam Kapı Kümesi (Complete Set of Gates)

Slaytta NAND kapısı kullanılarak AND, NOT ve OR kapılarının nasıl üretildiği gösterildi. Hoca bu kısmı geçti.

3. Lojik Fonksiyonların Gösterimi ve Çoğunluk Fonksiyonu

3.1 Dört Gösterim Biçimi

Bir lojik fonksiyon farklı şekillerde ifade edilebilir:

- **Doğruluk tablosu (truth table):** Girişlerin bütün kombinasyonları için çıkış yazılır.
- **Lojik ifade:** Örneğin $F = A \cdot B + \ldots$ biçiminde yazılan denklem.
- **Devre şeması:** Grafik biçim.
- **HDL kodu:** Verilog ve VHDL gibi dillerle yazılan tanım.

> **Ders sonrası not:** Hoca HDL'yi "hardware descriptor language" diye andı ve "gömülü sistem yazılımında kullanılan diller" arasında saydı. Açılımı büyük olasılıkla Hardware Description Language'dir. HDL büyük olasılıkla yazılım dili değil, donanım tanımlama dilidir; Verilog ve VHDL ile donanımın kendisi tarif edilir.

3.2 Üç Girişli Çoğunluk (Majority) Fonksiyonu

Bu gösterimleri bir örnekle görelim. Slayttaki tanım "output is 1 whether majority of inputs is 1" diyor: Girişlerdeki 1'lerin sayısı 0'lardan fazla ise çıkış 1 olacaktır. Bu fonksiyonu 3 giriş için gerçekleştirmek istiyoruz.

Tasarım şu sırayla ilerler. Önce problem tanımlanır, sonra doğruluk tablosu kurulur, sonra sadeleştirilmiş lojik ifade yazılır, en sonda devre şeması çizilir.

Doğruluk tablosu için A, B, C girişlerinin 8 kombinasyonunun hepsi yazılır. Çıkış tanıma göre doldurulur. 1 üreten ilk satır 011'dir, ardından 101, 110 ve 111 gelir.

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Bu tablo Karnaugh haritası üzerinden sadeleştirilirse şu ifade bulunur:

$$F = AB + AC + BC$$

Devre şeması üç tane AND kapısından ve bunların çıkışlarını toplayan bir OR kapısından oluşur.

3.3 Sadeleştirmeye Neden İhtiyaç Var?

Tablodaki dört satırı doğrudan yazsaydık, her biri 3 girişli 4 terim elde ederdik:

$$F = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$

Hoca bu formun neden tercih edilmediğini sordu. Sınıftan "daha az devre elemanı", "kapı tasarrufu", "daha ucuz" cevapları geldi. Hoca bunları onayladı: Aynı işi daha az devre elemanıyla yapabiliyorsak bu optimizasyonu donanım üzerinde de yapmamız gerekir. Sadeleştirilmiş devrenin daha anlaşılır olması da önemlidir, ama en önemlisi maliyettir.

4. Boole Cebri

Hoca slayttaki Boolean identities tablosunu satır satır gözden geçirdi. Tablo her özelliğin AND ve OR biçimini gösterir.

Özellik	AND biçimi	OR biçimi
Etkisiz eleman (identity)	$x \cdot 1 = x$	$x + 0 = x$
Tümleyen (complement)	$x \cdot \overline{x} = 0$	$x + \overline{x} = 1$
Değişme (commutative)	$xy = yx$	$x + y = y + x$
Dağılma (distribution)	$x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$
Involution	$\overline{\overline{x}} = x$	
Absorption	$x(x + y) = x$	$x + xy = x$
Birleşme (associative)	$(xy)z = x(yz)$	$(x + y) + z = x + (y + z)$
De Morgan	$\overline{xy} = \overline{x} + \overline{y}$	$\overline{x + y} = \overline{x}\,\overline{y}$

4.1 Etkisiz Eleman ve Tümleyen

Hoca birinci satırı bir soruyla okudu: Girişte bir x varsa ve çıkışta da x'i görmek istiyorsak diğer giriş ne olmalıdır? AND işleminde bu değer 1'dir, OR işleminde 0'dır. Sınıfın da söylediği gibi bu, matematikteki etkisiz elemana karşılık gelir.

Tümleyen satırında x ile tümleyeni birlikte kullanılır. x 0 ise tümleyeni 1, x 1 ise tümleyeni 0'dır. Yani bu iki değerin içinde mutlaka bir tane 1 ve bir tane 0 vardır. OR işleminde içerideki 1 çıkışı 1 yapar. AND işleminde içerideki 0 çıkışı 0 yapar.

Değişme özelliği, yer değiştirmenin sonucu değiştirmediğini söyler. Dağılma satırında ise slayt AND'in OR üzerine dağılımını gösterir.

4.2 Involution ve Absorption

> **Ders sonrası not:** Hoca involution satırında "x'in değilini aldığımızda yine x'i elde ediyoruz" dedi. Doğrusu: x'in değilinin değilini aldığımızda x'i elde ederiz, yani $\overline{\overline{x}} = x$. Tek değil, x'i değiştirir ($x = 0$ için $\overline{x} = 1$).

Hoca absorption için $x(x + y) = x$ eşitliğini gösterip bunun doğruluğunu nasıl söyleyebileceğimizi sordu ve yerine koyma yöntemini önerdi: x'e 0 ve 1 değerlerini verip sonuca bakın.

- $x = 0$ ise AND'in bir girişi 0 olduğu için ifade 0 olur. Sonuç x ile aynıdır.
- $x = 1$ ise parantez içindeki OR işleminin sonucu 1 olur. $1 \cdot 1 = 1$ olduğu için sonuç yine x ile aynıdır.

Soru: Bu, matematikteki yutan eleman değil mi? **Cevap:** Değil. Değer koyarak gidin, aynı sonucu elde ettiğinizi göreceksiniz.

> **Ek bilgi (derste anlatılmadı):** Karışıklık büyük olasılıkla iki adın benzemesinden kaynaklanıyor. Öğrencinin kastettiği yutan eleman büyük olasılıkla işlemin sonucunu tek başına belirleyen sabittir: $x \cdot 0 = 0$ ve $x + 1 = 1$. Absorption ise bir değişkenin kendisini içeren bir ifadeyi yutmasıdır: $x(x + y) = x$. Birincisinde sonuç sabittir, ikincisinde sonuç x'in kendisidir. Her iki eşitlik de bütün giriş değerleri için python ile doğrulandı.

4.3 De Morgan Kuralı

> **Hocanın vurgusu:** Bu bölümün belki de en önemli konusu De Morgan kuralıdır. Boole cebrinde çok işimize yarar ve sadeleştirmede çok kullanılır.

Bir AND işleminin değilini aldığımızda, x'in değili artı y'nin değili yazabiliriz. OR işlemi için de benzer biçimde yazılır:

$$\overline{xy} = \overline{x} + \overline{y}, \qquad \overline{x + y} = \overline{x}\,\overline{y}$$

Slaytın devamında çoğunluk fonksiyonunun diğer kapılarla, örneğin yalnız NAND kapılarıyla kurulmuş çözümü çizilmiştir.

> **Ek bilgi (derste anlatılmadı):** NAND ile temel kapılar şöyle yazılır: $\overline{x} = \overline{x \cdot x}$, $xy = \overline{\overline{xy}}$ ve $x + y = \overline{\overline{x}\,\overline{y}}$. Son eşitlik doğrudan De Morgan kuralıdır. Aynı yolla çoğunluk fonksiyonu $F = \overline{\overline{AB}\,\overline{AC}\,\overline{BC}}$ olarak yazılır; bu da üç NAND kapısının çıkışlarını dördüncü bir NAND'e vermek demektir. Bu eşitlikler python ile doğrulandı.

5. Karnaugh Haritası ile İndirgeme

Hoca Karnaugh haritasını üç değişkenli bir örnekle hatırlattı. Giriş değişkenleri x_1, x_2, x_3'tür. Doğruluk tablosunda her satır bir minterm'e karşılık gelir.

> **Hocanın vurgusu:** Ağırlıklara dikkat edin. Bu örnekte x_1 en yüksek anlamlı bittir, sıra x_1, x_2, x_3 biçimindedir.

Haritada yan yana gelen gözlerde yalnız bir bitin değişmesine izin vardır. Bu yüzden sıra $00, 01, 11, 10$ olmak zorundadır. Gözler buna göre numaralanır: İlk göz m_0'dır, en düşük anlamlı bit değişince komşusu m_1 olur ve bu şekilde m_7'ye kadar yerleştirilir.

$x_1 \backslash x_2x_3$	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

Sonra doğruluk tablosundaki değerler haritaya yerleştirilir ve oluşturulabilen en büyük gruplar kurulur. Slayttaki örnekte iki grup vardır. Yeşille çizilen büyük grup $\overline{x_3}$ terimini, diğer ikili grup $x_1\overline{x_2}$ terimini verir. İndirgenmiş fonksiyon şudur:

$$f = \overline{x_3} + x_1\overline{x_2}$$

Bu sonuçtan fonksiyonun mintermleri bulunabilir (ara adım eklendi): $\overline{x_3}$ terimi m_0, m_2, m_4, m_6 gözlerini, $x_1\overline{x_2}$ terimi m_4, m_5 gözlerini kapsar. Yani $f = \sum m(0, 2, 4, 5, 6)$ olur.

$x_1 \backslash x_2x_3$	00	01	11	10
0	1	0	0	1
1	1	1	0	1

> **Ek bilgi (derste anlatılmadı):** $\overline{x_3}$ grubu ilk ve son sütunu birlikte kapsar. Bu iki sütun haritada yan yana durmasa da komşudur, çünkü 00 ile 10 arasında da yalnız bir bit değişir (hesapla kontrol edildi). Yani harita kenarlardan dolanır.

6. Sayı Sistemleri ve İkiye Tümleyen

Bu kısım önemlidir, çünkü işlemcinin içinde bütün değerler binary tutulur ve bütün işlemler binary yapılır. Aritmetik lojik birim oluştururken de aritmetik işlemleri bu binary sayılar üzerinde yapacağız.

6.1 Onluk, İkilik ve Onaltılık Karşılıklar

Slayttaki tablo 0'dan 18'e kadar onluk sayıların ikilik ve onaltılık karşılıklarını verir. Hoca, bu tablonun genelde 15 veya 16'da kesildiğini, burada 18'e kadar götürülmesinin anlamlı olduğunu söyledi, çünkü 15'ten sonraki değişimi de görmekte fayda var.

Onluk	İkilik	Onaltılık	Onluk	İkilik	Onaltılık
0	00000	00	10	01010	0A
1	00001	01	11	01011	0B
2	00010	02	12	01100	0C
3	00011	03	13	01101	0D
4	00100	04	14	01110	0E
5	00101	05	15	01111	0F
6	00110	06	16	10000	10
7	00111	07	17	10001	11
8	01000	08	18	10010	12
9	01001	09			

15'in onaltılık karşılığı $0F$'tir. Ondan sonraki sayı olan 16'nın karşılığı ise 10'dur. 18, hocanın dediği gibi 16 artı 2 demektir.

> **Ders sonrası not:** Hoca "18 sayısının karşılığı 1F" dedi. Doğrusu $18 = 16 + 2 = 12_{16}$'dır. $1F_{16}$ ise 31'dir.

6.2 İşaretli Sayılar: Yalnız İkiye Tümleyen

İşaretli sayıları ifade etmek için farklı sayı sistemleri vardır. Hoca bunlarla kafamızı karıştırmayacağını söyledi: İşlemci içinde yalnız ikiye tümleyen (two's complement) kullanılır. Slaytın sağındaki tablo 4 bitlik işaretli sayıları $+7$ ile -8 aralığında verir.

İkilik	Değer	İkilik	Değer
0111	$+7$	1111	-1
0110	$+6$	1110	-2
0101	$+5$	1101	-3
0100	$+4$	1100	-4
0011	$+3$	1011	-5
0010	$+2$	1010	-6
0001	$+1$	1001	-7
0000	0	1000	-8

En yüksek anlamlı bit, yani 4 bitlik sayıda B_3, işaret bitidir. 0 ise sayı pozitif, 1 ise negatiftir. 4 bitle 16 farklı kombinasyon yazılabilir. Bir bit işarete ayrılınca geriye her işaret için 8 kombinasyon kalır. Bu yüzden aralık $+7$ ile -8 arasındadır.

Peki neden pozitif tarafta 8'e gidilemiyor? Çünkü 16 kombinasyondan biri 0'ı ifade eder. Bu sistemde 0'ın işaret biti 0 olduğu için pozitif taraftan bir değer kaybedilir. Böylece pozitif sayılar 7'ye, negatif sayılar 8'e kadar değer alabilir.

> **Ek bilgi (derste anlatılmadı):** Aynı sayım n bit için genelleştirilir: Aralık -2^{n-1} ile $2^{n-1} - 1$ arasındadır. $n = 4$ için bu, derste verilen -8 ile $+7$ aralığıdır. Formül 1 ile 12 bit arasındaki her genişlik için python ile doğrulandı.

6.3 İşaretli Bir Sayının Değerini Bulmak

Bir işaretli sayı verildiğinde ilk bakılacak şey en yüksek anlamlı bittir. Bu bit 0 ise değer doğrudan okunur: 0101'in 5, 0011'in $+3$ olduğu kolayca söylenir.

Negatif sayılarda ise işlem şöyledir. Örnek olarak 1011'e bakalım:

- **Verilen:** 1011, işaret biti 1 olduğu için negatif bir sayı.
- **İşlem:** Gerçek değeri bulmak için sayının tekrar ikiye tümleyeni alınır. Sonuç 0101'dir.
- **Ara sonuç:** $0101 = 5$.
- **Sonuç:** İşaret biti negatif dediği için $1011 = -5$.

  Ders sonrası not: Derste ara adım olarak "1010" söylendi. Doğrusu: 1011'in bitleri ters çevrilir (0100), sonra 1 eklenir (0101). 1010 ise -6'nın kodudur.

  Hocanın vurgusu: Öğrenciler bu konuda çok hata yapıyor. Farklı sayı sistemleri var, ama bilgisayarda onları kullanmıyoruz. İşaretli sayıları değerlendirirken aklınızda yalnız ikiye tümleyen olsun.

7. Multiplexer

Aradan önce hoca bundan sonraki kısmın neden önemli olduğunu söyledi: Kapılarla oluşturulmuş encoder, decoder, multiplexer gibi bloklar bizim yapı taşlarımızdır ve bunları işlemci tasarımında kullanacağız. Hoca bunları lojikte gördüğümüzü varsayarak önce fonksiyonlarını hatırlattı, sonra kullanım örneklerine geçti.

7.1 Çalışma Mantığı

Multiplexer'ın grafik sembolünün sol tarafında n tane veri girişi, ayrıca bu girişlerden hangisinin seçileceğini gösteren seçme (selection) girişleri vardır. Veri girişi sayısı n ise seçme ucu sayısı $\log_2 n$'dir. Bu yapıya n to 1 multiplexer denir. Türkçede seçici diye de anılır.

Hocanın örneği 8 giriştir: $8 = 2^3$ olduğu için $\log_2 8 = 3$ seçme ucu gerekir. Girişlere D_0'dan D_7'ye kadar diyelim. Seçme uçlarına 011 girilirse 3 numaralı giriş, yani D_3 seçilir ve yalnız o çıkışa gönderilir.

7.2 2'den 1'e Multiplexer

İki veri girişi (W_0, W_1), bir seçme ucu (S) ve bir çıkışı (F) vardır. Seçme ucu 0 ise W_0, 1 ise W_1 çıkışa gider. Devre şemasında birinci AND kapısı $S = 0$ iken W_0'ı, ikinci AND kapısı $S = 1$ iken W_1'i geçirir. Bir OR kapısı bu iki sonucu toplar:

$$F = \overline{S}\, W_0 + S\, W_1$$

7.3 4'ten 1'e Multiplexer

Dört veri girişi (W_0 - W_3) olduğu için iki seçme ucu (S_1, S_0) gerekir. Hoca, girişlerin ağırlıklarının, yani multiplexer'ın hangi girişine bağlandıklarının sembolde belirtildiğine dikkat çekti.

S_1	S_0	F
0	0	W_0
0	1	W_1
1	0	W_2
1	1	W_3

Devrede dört tane AND kapısı vardır. Her birinin üç girişi vardır: iki seçme ucunun ilgili kombinasyonu ve bir W girişi. Örneğin en üstteki AND kapısı, iki seçme ucu da 0 iken W_0'ı geçirir. Bir OR kapısı dört çıkışı toplar:

$$F = \overline{S_1}\,\overline{S_0}\, W_0 + \overline{S_1}\, S_0\, W_1 + S_1\, \overline{S_0}\, W_2 + S_1\, S_0\, W_3$$

7.4 Küçük Multiplexer'lardan Büyüğünü Kurmak

2'den 1'e multiplexer'lardan 4'ten 1'e: 3 tane 2'den 1'e multiplexer gerekir. Girişteki iki multiplexer'ın seçme ucu S_0'dan gelir. Her ikisinde de seçme ucu 0 iken üstteki, 1 iken alttaki veri seçilir. Çıkış katmanındaki multiplexer'ın seçme ucu ise daha yüksek anlamlı olan S_1'den gelir. $S_1 = 0$ ise üst sıradaki, $S_1 = 1$ ise alt sıradaki multiplexer'ın çıkışı devrenin çıkışına aktarılır.

4'ten 1'e multiplexer'lardan 16'dan 1'e: Mantık aynıdır. Dört seçme ucu vardır (S_3, S_2, S_1, S_0). Düşük anlamlı S_1, S_0 uçları girişteki 4 multiplexer'ın seçme uçlarına bağlanır. Yüksek anlamlı S_3, S_2 uçları ise çıkışı oluşturan multiplexer'ın seçme uçlarına bağlanır. Toplam 5 tane 4'ten 1'e multiplexer kullanılır (ara adım eklendi).

8. Decoder

8.1 Çalışma Mantığı

Decoder'ın yaptığı iş kod çözmedir. Slayttaki tanım "kodlanmış bilgiyi çözmek amacıyla kullanılır" der. Sembolün solunda girişler, sağında çıkışlar vardır. n tane giriş ve bir enable girişi vardır, çıkış sayısı 2^n'dir.

Enable ucu 0 iken decoder devre dışı gibidir ve bütün çıkışlar 0 olur. Enable ucu 1 iken girişte belirtilen değerle ilgili tek çıkış 1 yapılır, diğer bütün çıkışlar 0 olur.

8.2 2'den 4'e Decoder

E	W_1	W_0	Y_0	Y_1	Y_2	Y_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Girişte 00 varsa 0 sayısı kodlanmış demektir ve Y_0 çıkışı 1 olur. 01 varsa 1 sayısı kodlanmıştır ve Y_1 çıkışı 1 olur. 2 ve 3 için de aynı şekilde Y_2 ve Y_3 aktif olur. Yani iki biti dört bite genişletmiş, hangi sayının kodlandığını çıkışta göstermiş oluruz.

> **Hocanın vurgusu:** Grafik sembol çok önemlidir ve çizimlerde bu sembole uygun çizim beklenir. Solda girişler, sağda çıkışlar olacak. Her giriş için ağırlık mutlaka yazılacak. W yazmak gerekmez, ama hangisinin 1, hangisinin 0, hangisinin yüksek anlamlı olduğu belirtilmelidir. Aynı şey çıkışlar için de geçerlidir. Bunlar yazılmazsa çözüm "hiçbir anlam ifade etmez".

8.3 2'den 4'e Decoder'lardan 4'ten 16'ya Decoder

4 giriş vardır: W_0, W_1, W_2, W_3 ve en yüksek anlamlı olan W_3'tür. Düşük anlamlı iki bit olan W_0, W_1, 4 tane decoder'ın her birinin W_0, W_1 girişine ortak bağlanır. Dikkat edilecek nokta enable uçlarıdır: Bu dört decoder'ın enable uçları, girişe yerleştirilen beşinci bir 2'den 4'e decoder'ın çıkışlarından gelir. Bu beşinci decoder'ın girişleri W_3, W_2'dir.

W_3W_2	Enable alan decoder	Çıkış aralığı
00	En üstteki	Y_0 - Y_3
01	İkinci	Y_4 - Y_7
10	Üçüncü	Y_8 - Y_{11}
11	En alttaki	Y_{12} - Y_{15}

Hocanın izlediği durumlar şöyledir. $W_3W_2 = 00$ ise ilk decoder'ın Y_0 çıkışı aktif olur. En üstteki decoder enable olur, diğerlerinin enable uçları 0 olduğu için bütün çıkışları 0'dır. Üstteki

decoder'ın hangi çıkışının 1 olacağına W_1, W_0 karar verir. $W_3W_2 = 01$ ise Y_1 aktif olur ve ikinci sıradaki decoder enable olur. $W_3W_2 = 11$ ise Y_3 aktif olur ve en alttaki decoder seçilir. Yine onun hangi çıkışının seçileceğine W_1, W_0 karar verir. Toplam 5 tane 2'den 4'e decoder kullanılır.

Soru: 4'ten 16'ya decoder için 2'den 4'e decoder mu kullanmak gerekiyor, yoksa doğrudan 4'ten 16'ya decoder var mı? **Cevap:** Kesinlikle var. 5'ten 32'ye de var. Bu örnek yalnız alıştırma olsun diye verilmiş. Bunların hepsi mevcuttur, ama birbirlerinden türetilebildiğini görmek faydalıdır.

Hocanın eklediği bir örnek de şudur: 3'ten 8'e decoder'ın iç devresini kendimiz çizmek istesek 3 giriş, 1 enable ucu ve 8 tane 4 girişli AND kapısı gerekir. Her AND kapısının çıkışı bir Y çıkışını oluşturur.

9. Demultiplexer

Hoca bu noktada iki önemli yapı taşını öğrendiğimizi, şimdi bunların terslerine geçeceğimizi söyledi. Demultiplexer, sanki multiplexer'ın grafiği ters çevrilmiş gibi düşünülebilir. Solda tek bir giriş, sağda çıkışlar, yukarıda seçme uçları vardır. Çıkış sayısı n ise seçme ucu sayısı $\log_2 n$'dir. Yaptığı iş, girişin hangi çıkışa gönderileceğine karar vermektir. Yani multiplexer'ın tersi biçimde çalışır.

Slaytta 1'den 4'e demultiplexer'ın tablosu verilmiştir. A bir bitlik girişi gösterir. Seçme değeri 0 ise Q_0 çıkışında girişte ne varsa o görünür. Diğer seçme değerleri de aynı mantıkla ilgili çıkışı seçer (ara adım eklendi).

Seçme değeri	A'nın göründüğü çıkış
0	Q_0
1	Q_1
2	Q_2
3	Q_3

Hoca demultiplexer'ın çok kullanılmadığını, en çok kullandığımız yapının multiplexer olduğunu söyledi. Yalnız yapıyı anlamak için göstermekte fayda var.

10. Encoder

10.1 Çalışma Mantığı

Encoder'ın 2^n girişi ve n çıkışı vardır. Adı 2^n to n encoder olarak geçer, örneğin 8'den 3'e encoder. 2^n bitle verilen ifadeyi n bitle kodlar.

10.2 4'ten 2'ye Encoder

Girişler W_3'ten W_0'a, çıkışlar Y_1, Y_0 diye adlandırılmıştır.

> **Hocanın vurgusu:** Girişte yalnız bir tane 1 olabilir. Bu çok önemli bir kuraldır. Aynı anda birden fazla giriş kesinlikle 1 olamaz.

$W_3W_2W_1W_0$	Y_1Y_0
0001	00
0010	01
0100	10
1000	11

Mantık şudur: Hangi kanaldan 1 geldiyse o kanalın numarasını kodlarsınız. W_0'dan geldiyse 0, W_1'den geldiyse 1, W_3'ten geldiyse 3 kodlanır.

Tablo kurulduktan sonra çıkış denklemlerini yazmak kolaydır. Hoca bunu sonradan bir soru üzerine gösterdi. Y_0 çıkışının 1 olduğu iki durum vardır: W_1 ve W_3'ün 1 olduğu durumlar. Yani Y_0, W_1 ile W_3'ün OR'lanmasıyla üretilir. Aynı okuma Y_1 için de yapılır (ara adım eklendi):

$$Y_0 = W_1 + W_3, \qquad Y_1 = W_2 + W_3$$

Soru: Bu encoder fiziksel olarak gerçekleştirildiğinde, girişlerden birden fazlasına 1 verirsek nasıl bir sonuç alırız? **Cevap:** Girişte yalnız bir tanesinin 1 olmasını sizin garantilemeniz gerekir. Başka bir durumda doğru sonuç üretme garantisi yoktur.

> **Ders sonrası not:** Soruyu soran öğrenci bu yapıya "2'den 4'e enkoder" dedi ve bu hemen düzeltilmedi. Sözü geçen yapı 4'ten 2'ye encoder'dır.

Soru: Bu encoder için Karnaugh haritası kurarken, örneğin W_3 ve W_2'nin birlikte 1 olduğu 1100 durumuna Y_1 için ne yazacağız, X mi? **Cevap:** Burada Karnaugh haritasına hiç gerek yok, o kadar basit. Aynı anda yalnız bir kanaldan 1 geleceğini biliyorsunuz, öbür durumlar hiç olmayacak. Bu yüzden fonksiyonu tablodan yazıp grafiğini çizmek daha kolaydır.

11. Priority Encoder

11.1 Yapısı ve Doğruluk Tablosu

Priority encoder'ın da 2^n girişi (W_0 - W_{2^n-1}) ve n çıkışı (Y_0 - Y_{n-1}) vardır. Ek olarak bir Z çıkışı üretir. Z, valid yani geçerlilik bitidir ve geçerli veri olup olmadığını gösterir. 4'ten 2'ye örnekte W_3 en yüksek, W_0 en düşük önceliklidir.

W_3	W_2	W_1	W_0	Y_1	Y_0	Z
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

Satırları hoca şöyle okudu. Girişlerde hiç 1 yoksa $Z = 0$ olur, yani geçerli veri yoktur. Bu durumda Y_1, Y_0 çıkışlarında anlamlı bir değer aranmaz, çünkü önce Z'ye bakılır. İkinci satırda en az bir tane 1 vardır, $Z = 1$ olur. 1, W_0'dan geldiği için 0 kodlanır. Sonraki satırda 1, W_1'den gelmiştir ve 01 kodlanır. En alt satırda 1, W_3'ten gelmiştir ve 3 kodlanır.

Peki tablonun sağ tarafındaki bitler neden gösterilmemiş? Çünkü yüksek öncelikli bir birimden 1 gelmişse daha düşük öncelikli olan sağdaki girişler hiç dikkate alınmaz, 0 da olsa 1 de olsa fark etmez. Örneğin W_2'den 1 gelmişse, W_1 ve W_0 ne olursa olsun, W_2'ye bakılır.

11.2 Kullanım Alanı: Kesme İstekleri

Priority encoder, işlemciye gelen kesme (interrupt) isteklerini tanımlarken kullanılır. Hoca bir şema üzerinden anlattı: Ortada bir CPU, çevresinde D_0, D_1, D_2, D_3 birimleri var. Bunlar işlemciye veri gönderen klavye, fare, diskten okuma veya USB portları gibi birimler olabilir. Birimlerin öncelik sırası sistemde baştan tanımlıdır.

Hocanın örneğine göre düşük öncelikli birimler yavaş çalışan birimlerdir. Klavye iyi bir örnektir: Klavyenin veri gönderme hızı insanın yazma hızına bağlıdır. Saniyede basabildiğimiz tuş sayısı sınırlı olduğu için sistemin en yavaş birimidir. Yüksek öncelikli birim ise en yüksek hızda çalışan birimdir ve ona daha öncelikli hizmet vermek gerekir.

Bu şemada en düşük öncelikli birim D_0, tablodaki W_0'a karşılık gelir. En yüksek öncelikli birim ise D_3, tablodaki W_3'e karşılık gelir. (Hoca önce D_3 için "en düşük" dedi, hemen ardından düzeltti.)

- Yalnız W_0'dan kesme isteği gelirse Y_1Y_0 çıkışı, isteğin 0 numaralı kanaldan geldiğini gösterir.

- 3 numaralı birimden istek gelirse, öteki birimlerden 0 da gelse 1 de gelse önemi yoktur. 3 numaralı kanal kodlanır, bu birime cevap verilir ve veri transferi yapması sağlanır.

  Hocanın vurgusu: Priority encoder, işlemci tasarımında göreceğimiz yapı taşlarından biridir.

11.3 Soru-Cevaplar

Soru: Bu priority encoder fiziksel olarak gerçekleştirildiğinde dört girişe de 0 verirsek Y_1 ve Y_0 nasıl sonuç verir? **Cevap:** Ne ürettiğinin önemi yok. CPU örneğini düşünün: Bütün girişlerin 0 olması, hiçbir birimden kesme isteği gelmemesi demektir. Bu durumda geçerlilik biti 0 olur ve işlemciye istek olmadığını söyler. Önce Z bitine bakılır, istek yoksa Y_1 ve Y_0 değerlendirilmez. Devreyi tasarlarken 0 yazdıysanız 0, 1 yazdıysanız 1 verir, ama bu değer kullanılmaz.

Soru: Yani başta Y_1 ve Y_0'a bir değer atıyoruz, değil mi? **Cevap:** Evet. Bu bir doğruluk tablosu. Devreyi tasarlayabilmek için örneğin ikisine de 0 yazarım.

Soru: 0101 ile 0110 eşit önceliğe mi sahiptir? **Cevap:** Her ikisinde de W_2 kanalı aktif olur, çünkü sağdaki iki bit önemli değildir. W_1, W_0 ne olursa olsun W_2'ye hizmet verilir ve çıkışta 10 üretilir. Diğer iki birimden de istek gelebilir, ama yüksek öncelik W_2'dedir.

> **Ders sonrası not:** Bu cevapta hoca "W_3 1 olduğunda" dedi. Doğrusu: $W_3 = 0$ ve $W_2 = 1$ olduğunda W_1, W_0 önemsizdir. $W_3 = 1$ olsaydı W_3'e hizmet verilirdi (hocanın tablosunda da böyledir).

11.4 Tabloyu Karnaugh Haritasına Taşımak

Hoca bu doğruluk tablosunu Karnaugh haritasına taşıyıp Y_1, Y_0 ve Z çıkışları için indirgenmiş fonksiyonları yazmayı alıştırma olarak verdi. Burada dikkat edilecek bir nokta vardır: Tabloda 5 satır var, oysa 4 değişken için 16 kombinasyon vardır. Haritada 16 gözün hepsi doldurulmalıdır. Tablodaki tireli gözler "ne olursa olsun" (don't care) anlamındadır: Hoca "şu gözün tire geçmesi, burası 0 da olsa 1 de olsa bu durum bu sonucu üretecek demektir" dedi.

Soru: Karnaugh haritasına aktarırken bunlar don't care mi olacak? **Cevap:** Hayır. Girişte don't care kullanamazsınız. Girişte bütün kombinasyonları göreceksiniz ve bu kombinasyonlar için tablodaki çıkışı üreteceksiniz. Yani bütün girişleri tabloda değil, Karnaugh haritasında dolduracaksınız. Basit encoder'ın aksine burada diğer kanallardan 0 ya da 1 gelebilir, bu yüzden hoca bu tabloda Karnaugh haritası çizilmesini özellikle istedi.

Yani tireli bir satır haritada birden çok göze açılır ve bu gözlerin hepsine aynı çıkış yazılır (ara adım eklendi): 0000 ve 0001 birer göz, $001X$ iki göz, $01XX$ dört göz, $1XXX$ sekiz göz, toplam 16 göz.

> **Ders sonrası not:** Bu alıştırmanın sonucu derste verilmedi. Hesapla bulunan indirgenmiş denklemler şunlardır: $Y_1 = W_3 + W_2$, $Y_0 = W_3 + \overline{W_2}\,W_1$, $Z = W_3 + W_2 + W_1 + W_0$. Bu denklemler 16 girişin hepsinde tabloyla aynı sonucu verir. 0000 girişinde Y_1 ve Y_0 için 00 üretirler, yani hocanın "ikisine de 0 yazarım" seçimiyle uyumludur. Çıkışı gerçekten serbest olan tek satır 0000'dır.

12. Soru 1: Decoder ve AND-OR Dizisi ile 8'den 1'e Multiplexer

Bundan sonra hoca, ders kitabının bölüm sonundan dört problem çözdü. Çözümler önce öğrencilerden istendi, sonra birlikte tahtada (whiteboard uygulaması) yapıldı.

Problem: "Design an 8-to-1-line multiplexer using a 3-to-8-line decoder and an 8×2 AND-OR." Yani elimizdeki malzeme bir 3'ten 8'e decoder ve bir 8×2 AND-OR dizisidir.

8×2 AND-OR dizisinin anlamı şudur: Her biri 2 girişli 8 tane AND kapısı vardır ve bu kapıların çıkışları bir OR kapısıyla toplanır.

Soru: 3'ten 8'e decoder'ı istediğimiz kadar kullanabilir miyiz? **Cevap:** Yalnız bir kere kullanmak yetiyor.

Hatırlatma. Hoca çözümden önce 4'ten 1'e multiplexer'ın iç yapısını açtı. 4 tane 3 girişli AND kapısı vardır: $\overline{S_1}\,\overline{S_0}\,W_0$, $\overline{S_1}\,S_0\,W_1$, $S_1\,\overline{S_0}\,W_2$ ve $S_1\,S_0\,W_3$. Bir OR kapısı bunları toplar. Buradaki

seçme kombinasyonlarını üreten kısım ile "AND'le ve topla" kısmını ayrı görürsek, verilen iki malzemenin ne işe yarayacağı ortaya çıkar. Decoder ise girişteki değere karşılık gelen kanalı işaretler: Örneğin $W_1W_0 = 10$ olduğunda Y_2 kanalı işaretlenir.

Çözüm. Sınıftan gelen doğru çözümü hoca şöyle çizdi:

- S_2, S_1, S_0 seçme uçları, ağırlık sırasıyla 3'ten 8'e decoder'ın 2, 1 ve 0 numaralı girişlerine bağlanır. S_2 en yüksek anlamlıdır.
- Decoder'ın her Y_i çıkışı, W_i veri girişiyle bir AND kapısına girer: Y_0 ile W_0, Y_1 ile W_1, ..., Y_7 ile W_7.
- AND kapılarının çıkışları OR kapısında toplanır. OR çıkışı multiplexer'ın F çıkışıdır.

$$F = Y_0W_0 + Y_1W_1 + \cdots + Y_7W_7$$

Decoder'ın çıkışları burada izin görevi görür. Seçilen kanalın Y_i'si 1, diğerleri 0 olduğu için yalnız o W_i çıkışa geçer. Bu devre bütün seçme ve veri girişi kombinasyonlarında python ile sınandı ve 8'den 1'e multiplexer davranışını verdi.

> **Ders sonrası not:** Hoca "bu çıkışların her biri decoder'ın izin girişlerini oluşturacak" dedi. Doğrusu: Decoder'ın çıkışları, AND kapılarının izin girişlerini oluşturur.

Hoca, girişlere A, B, C de denebileceğini, burada ağırlıklı adlar verdiğini ama her zaman böyle kullanılmadığını belirtti. Bu yüzden girişlerin decoder'a nasıl bağlandığı mutlaka belirtilmelidir.

13. Soru 2: Decoder ve OR Kapılarıyla Üç Fonksiyon

Problem: X, Y, Z girişleriyle üç bağımsız fonksiyon verilmiştir. Devre decoder ve harici OR kapılarıyla tasarlanacaktır.

$$F_1 = \overline{X+Z} + XYZ, \qquad F_2 = \overline{X+Z} + \overline{X}YZ, \qquad F_3 = X\overline{Y}Z + \overline{X+Z}$$

Hoca önce çözüm yolunu çizdi: Fonksiyonlar doğruluk tablosuna yerleştirilir, sonra her fonksiyonun hangi satırlarda 1 olduğuna bakılır. Tek bir decoder kullanılacak ve 3 çıkış için 3 OR kapısı gerekecek.

13.1 Doğruluk Tablosu

Tablo öğrencilerle birlikte doldurduldu. F_1 için satır satır gidildi:

- 000: $X + Z = 0$, tersi 1 olduğu için $F_1 = 1$.
- 001: $X + Z = 1$, tersi 0. $XYZ = 0$. Sonuç $F_1 = 0$.
- 010: $X + Z = 0$, tersi 1. Bir terimin 1 olması yeterlidir, $F_1 = 1$.
- 011: $X + Z = 1$, tersi 0. $XYZ = 0$. Sonuç $F_1 = 0$.
- 100: Sağdaki çarpım 0. $X + Z = 1$, tersi 0. Sonuç $F_1 = 0$.
- 101 ve 110: Aynı şekilde iki terim de 0, $F_1 = 0$. Bir öğrencinin dediği gibi, sağdaki çarpım yalnız 111'de 1 gelir.
- 111: $XYZ = 1$ olduğu için doğrudan $F_1 = 1$.

F_2 için bir öğrenci kestirme yol kullandı: $\overline{X+Z}$ terimi X ve Z'nin 0 olduğu 000 ve 010 satırlarında 1'dir. İkinci terim 011 satırında 1 verir. Geri kalanlar 0'dır. F_3 için yine 000 ve 010 satırlarında 1 gelir, $X\overline{Y}Z$ terimi de 101 satırında 1 verir.

X	Y	Z	Satır	F_1	F_2	F_3
0	0	0	0	1	1	1
0	0	1	1	0	0	0
0	1	0	2	1	1	1
0	1	1	3	0	1	0
1	0	0	4	0	0	0
1	0	1	5	0	0	1

X	Y	Z	Satır	F_1	F_2	F_3
1	1	0	6	0	0	0
1	1	1	7	1	0	0

Sonuçta $F_1 = \sum m(0, 2, 7)$, $F_2 = \sum m(0, 2, 3)$, $F_3 = \sum m(0, 2, 5)$ olur.

13.2 Devre

3 giriş olduğu için 3'ten 8'e decoder kullanılır. Hoca burada ağırlıklara yine dikkat çekti: Soruda S_0, S_1 gibi adlar yoktur, ama tablonun en alt satırına 7 dediğimize göre X en yüksek anlamlıdır. Decoder'ın içine 0, 1, 2 ağırlıkları yazılmışsa 2 numaralı giriş en yüksek anlamlı giriştir. Bu yüzden X 2'ye, Y 1'e, Z 0'a bağlanır.

> **Hocanın vurgusu:** Decoder kullanırken iç yüzüne ağırlıkları mutlaka koyun ve dışarıdaki değişkenleri bu ağırlıklara göre bağlayın.

Decoder'ın 8 çıkışından yalnız gerekenler kullanılır:

- F_1: 0, 2 ve 7 numaralı çıkışlar bir OR kapısıyla toplanır.
- F_2: 0, 2 ve 3 numaralı çıkışlar bir OR kapısıyla toplanır.
- F_3: 1 olan çıkışlar, yani 0, 2 ve 5 numaralı çıkışlar, bir OR kapısıyla toplanır.

14. Soru 3: 8'den 1'e Multiplexer ve Bir Inverter

Problem: "Implement the following Boolean function with an 8-to-1-line multiplexer and a single inverter with variable D as its input."

$$F(A, B, C, D) = \sum m(2, 4, 6, 9, 10, 11, 15)$$

8'den 1'e multiplexer'ın 3 seçme ucu vardır, ama fonksiyonun 4 girişi vardır. Bir giriş açıkta kalır. Soru bize çözümü söylüyor: D seçme ucu olarak kullanılmayacak, multiplexer'ın veri girişlerinde kullanılacak. Bunun için bir tane NOT kapısı kullanma hakkımız var. Seçme uçlarına ise A, B, C bağlanacak.

Ağırlık sırası burada da önemlidir. A en yüksek ağırlıklı bit olduğu için 2 numaralı seçme ucuna A, 1 numaralıya B, 0 numaralıya C bağlanır.

Önce 16 satırlık doğruluk tablosu yazılır. Hoca bu tabloyu uzun bulsa da başka türlü açıklanamayacağını söyledi. Sonra tablo, ABC değerine göre ikişer satırlık gruplara ayrılır. Her grupta şu soru sorulur: F çıkışı ile D arasında bir ilişki var mı, varsa nedir?

ABC	Mintermler	$D = 0$ iken F	$D = 1$ iken F	MUX girişi
000	0, 1	0	0	0
001	2, 3	1	0	$\overline{D}$
010	4, 5	1	0	$\overline{D}$
011	6, 7	1	0	$\overline{D}$
100	8, 9	0	1	D
101	10, 11	1	1	1
110	12, 13	0	0	0
111	14, 15	0	1	D

Tablonun okunuşu şöyledir:

- $ABC = 000$: D ne olursa olsun $F = 0$. Bu yüzden 0 numaralı giriş 0'a (toprağa, ground) bağlanır.
- $ABC = 001$: F, D'nin tümleyenidir. Sınıf bunu hemen söyledi. D bir inverter'dan geçirilir ve 1 numaralı girişe bağlanır.
- $ABC = 010$ ve 011: Yine $\overline{D}$. Aynı inverter çıkışı 2 ve 3 numaralı girişlere de bağlanır.

- $ABC = 100$: F, D ile aynıdır. D ucu 4 numaralı girişe bağlanır.
- $ABC = 101$: F, D'den bağımsız olarak 1'dir.
- $ABC = 110$: F, D'den bağımsız olarak 0'dır.
- $ABC = 111$: F, D ile aynıdır. 7 numaralı giriş D'den alınır.

$\overline{D}$ için tek inverter yeterlidir, bu da sorudaki koşulu sağlar. Devre, 16 giriş kombinasyonunun hepsinde verilen fonksiyonu üretir (python ile doğrulandı).

> **Hocanın vurgusu:** Çok üstünde durduğum konulardan biri: Multiplexer'ın içindeki ağırlıkları hem seçme uçları hem de veri uçları için mutlaka belirtmelisiniz.

15. Soru 4: 4'ten 1'e Multiplexer ve Harici Kapılar

Problem: "Implement the Boolean function with a 4-to-1-line multiplexer and external gates. Connect inputs A and B to the selection lines. The input requirements for the four data lines will be a function of variables C and D."

$$F(A, B, C, D) = \sum m(1, 3, 4, 11, 12, 13, 14, 15)$$

Hoca bu sorunun bir öncekinden biraz daha zor olduğunu söyledi. Yine dört girişli bir fonksiyon var, ama multiplexer küçüldüğü için veri girişlerine artık yalnız D değil, C ve D'nin fonksiyonları gelecek. "Gates" dendiği için harici kapı sayısı tek değildir.

Seçme uçlarının ağırlıkları içeride yazılıdır. Yüksek anlamlı uca A, düşük anlamlı uca B bağlanır. Sonra tablo AB değerine göre dörder satırlık gruplara ayrılır ve her grupta F ile C, D arasındaki ilişki aranır.

AB	Mintermler	CD: 00, 01, 10, 11 iken F	MUX girişi
00	0, 1, 2, 3	0, 1, 0, 1	D
01	4, 5, 6, 7	1, 0, 0, 0	$\overline{C}\,\overline{D} = \overline{C + D}$
10	8, 9, 10, 11	0, 0, 0, 1	CD
11	12, 13, 14, 15	1, 1, 1, 1	1

Satırlar şöyle okunur:

- $AB = 00$: F doğrudan D'ye eşittir. 0 numaralı girişe D bağlanır.
- $AB = 01$: Yalnız $CD = 00$ için 1 olmuştur. Hoca bir yol önerdi: C ve D girişleriyle F için küçük bir doğruluk tablosu kurup bu 1'i sağlayan fonksiyonu yazın. Sonuç $\overline{C}\,\overline{D}$'dir. De Morgan ile $\overline{C + D}$ yazılır, yani 1 numaralı giriş bir NOR kapısıyla üretilir.
- $AB = 10$: Yalnız $CD = 11$ için 1 olmuştur. Sınıftan da söylendiği gibi C ile D bir AND kapısından geçirilip 2 numaralı girişe verilir.
- $AB = 11$: F, C ve D'den bağımsız olarak 1'dir. 3 numaralı girişe 1 bağlanır.

Soru: Hoca $AB = 01$ durumu için önce "NAND ile de yapılabilir" deyince bir öğrenci sordu: NAND'de üç tane 1 olmuyor mu? **Cevap:** Doğru, NOR olacak. NAND'in çıkışında üç tane 1 vardır, burada ise yalnız tek bir 1 lazımdır.

Hesapla kontrol: C, D kombinasyonları $00, 01, 10, 11$ iken NAND çıkışı $1, 1, 1, 0$, NOR çıkışı $1, 0, 0, 0$'dır. Gereken tablo NOR'unkidir. Devre, 16 giriş kombinasyonunun hepsinde verilen fonksiyonu üretir (python ile doğrulandı).

Dersin Toparlaması

Bu derste lojik konularının hızlı bir tekrarı yapıldı. AND, OR ve özellikle XOR kapılarının tek cümlelik tanımları, XOR'un even parity üreteci olması, lojik fonksiyonların dört gösterimi, Boole cebri kuralları (en başta De Morgan), Karnaugh haritasıyla indirgeme ve işaretli sayılarda ikiye tümleyen gözden geçirildi. Ardından işlemci tasarımında kullanılacak yapı taşları görüldü: multiplexer, decoder, demultiplexer, encoder ve priority encoder. Küçük bloklardan

büyüklerinin nasıl kurulduğu ve priority encoder'ın kesme isteklerindeki rolü anlatıldı. Dört problemle de lojik fonksiyonların decoder veya multiplexer yardımıyla nasıl gerçekleştirildiği gösterildi. Bütün bu çözümlerin ortak uyarısı, giriş ve çıkış ağırlıklarının mutlaka yazılmasıdır.

Lojik konularının tekrarı ilk iki üç haftada tamamlanacak ve hızlı ilerlenecek. Bu konuları unutanların lojik kitaplarına bakarak eksiklerini hızla tamamlaması gerekiyor. Sıradaki konu ardışıl lojik devreler, flip-floplar ve bunlarla yapılan tasarımlardır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.